

CMPS 6610 Problem Set 03

Answers

Name: Khoa Le

Place all written answers from `problemset-03.md` here for easier grading.

- 1a.

Done in `main.py` as `isearch`. I use `iterate` with a boolean accumulator:

```
return iterate(lambda found, element: found or element == x, False, L)
```

The accumulator means “have I seen x so far”. I start at `False` because that is the right answer for an empty list and it is also the identity of `or`. No preprocessing needed.

- 1b.

$$W(n) = W(n-1) + O(1), \quad S(n) = S(n-1) + O(1)$$

$$W(n) \in O(n), \quad S(n) \in O(n)$$

Work: `iterate` looks at every element once and does one comparison each, so it is linear.

Span: the span equals the work here because `iterate` is a chain. Step i needs the accumulator from step $i-1$, so nothing can run in parallel.

- 1c.

Done in `main.py` as `rsearch`. I map first, then reduce:

```
return reduce(lambda a, b: a or b, False, [element == x for element in L])
```

I map each element to `element == x` first because `reduce` returns `a[0]` directly when the list has one element, so it never applies `f` there. If I put the comparison inside `f` instead, `rsearch([5], 3)` would return 5 instead of a boolean. After the map, `or` is a proper associative operator with identity `False`, which is what reduce needs.

- 1d.

$$W(n) = 2W(n/2) + O(1), \quad S(n) = S(n/2) + O(1)$$

$$W(n) \in O(n), \quad S(n) \in O(\log n)$$

Work: reduce splits in half and the combine is only $O(1)$, so the leaves dominate and it is linear. The map before it is also $O(n)$, so it doesn't change anything.

Span: the two recursive calls are independent and run in parallel, so the critical path follows only one of them. That gives a tree of depth $\log n$ with $O(1)$ per level, instead of the chain of length n in 1b.

- 1e.

$$W(n) = W(n/3) + W(2n/3) + O(1), \quad S(n) = S(2n/3) + O(1)$$

$$W(n) \in O(n), \quad S(n) \in O(\log n)$$

Work: the two subproblem sizes still add up to n and the combine is still $O(1)$, so it is still leaf-dominated and still linear. The uneven split does not matter for work.

Span: the two calls run in parallel so the critical path follows the bigger one, the $2n/3$ branch. That shrinks by a factor of $3/2$ each level, so the depth is $\log_{3/2} n$, which is still $O(\log n)$.

So **ureduce** has the same work and span as **reduce**. The only difference is the constant, since $\log_{3/2} n \approx 1.71 \log_2 n$, so the critical path is about 71% longer.

- **2a.**

I cannot just sort A because that would destroy the order. So I tag each element with its index first, sort by value, keep only the first copy of each value, then sort back by index and drop the tags.

```
dedup A =
  let
    T = ⟨ (A[i], i) : 0 ≤ i < |A| ⟩
    S = sort_≤_v T   where (x, i) ≤_v (y, j) ≡ x < y ∨ (x = y ∧ i ≤ j)
    F = ⟨ S[k] : 0 ≤ k < |S| | k = 0 ∨ π1(S[k]) ≠ π1(S[k-1]) ⟩
    U = sort_≤_p F   where (x, i) ≤_p (y, j) ≡ i ≤ j
  in
    ⟨ π1(u) : u ∈ U ⟩
end
```

After sorting, all copies of the same value sit next to each other and are ordered by index, so the copy I keep is the one with the smallest index, which is the first occurrence in A . Sorting by index at the end puts them back in input order.

$$W(n) \in O(n \log n), \quad S(n) \in O(\log^2 n)$$

Work: the tabulate, filter and map are all $O(n)$, so the two merge sorts dominate at $O(n \log n)$.

Span: same reason, merge sort has span $O(\log^2 n)$ and everything else is only $O(\log n)$.

- **2b.**

Now there are $m + 1$ lists of n elements, so $N = (m + 1)n$ total. Order doesn't matter anymore, so I can drop the tagging and the second sort. I just flatten everything into one list, sort it, and keep an element only if it differs from the one before it.

```
multidedup A =
  let
    B = flatten A
    S = sort_≤ B
  in
    ⟨ S[k] : 0 ≤ k < |S| | k = 0 ∨ S[k] ≠ S[k-1] ⟩
end
```

$$W(N) \in O(N \log N) = O(mn \log(mn)), \quad S(N) \in O(\log^2 N) = O(\log^2(mn))$$

Work: flatten and the filter are $O(N)$, so the sort dominates again.

Span: the sort dominates again, $O(\log^2 N)$.

Comparing to 2a: it is really the same algorithm with $m = 0$. The work goes up by a factor of about $m + 1$, which we can't avoid since every element has to be looked at. But the span only goes from $O(\log^2 n)$ to

$O(\log^2(mn))$, and $\log(mn) = \log m + \log n$, so the latency barely grows. That means this problem parallelizes very well, adding more machines costs almost nothing in span as long as we have the processors.

- **2c.**

Yes, several of them are useful.

`sort` is the important one. Without it I would have to compare every pair to find duplicates, which is $O(n^2)$ work. Sorting puts equal elements next to each other, so checking for a duplicate becomes a constant-time test against my neighbor. That is what gets the work down to $O(n \log n)$.

`tabulate` and `map` handle the index tagging and untagging in 2a, $O(n)$ work and $O(\log n)$ span.

`filter` removes the duplicates in one parallel pass, $O(n)$ work and $O(\log n)$ span. Its predicate is only constant-time because I sorted first.

`flatten` is what makes 2b work, it turns the $m + 1$ lists into one list in $O(N)$ work and $O(\log N)$ span so I can reuse the single-list solution.

`reduce` would let me merge the $m + 1$ partial results in a tree of depth $O(\log m)$ instead of a chain of length $O(m)$. This works because set union is associative with the empty list as identity, which is what `reduce` requires.

`scan` does not help. Whether $A[i]$ is a duplicate depends on what set of things came before it, and that running set grows to size $O(n)$, so the combine is not constant-time and a scan would end up $O(n^2)$ work. Sorting avoids this by turning a global dependency into a local one.

`iterate` solves it correctly with a hash set and only $O(n)$ work, which is actually better than sorting. But its span is $O(n)$ since it is a chain, so it is useless for 2b where the whole point is low latency.

- **3a.**

Done in `main.py` as `parens_update` and `parens_match_iterative`. I keep one counter of how many parens are currently open. (adds 1,) subtracts 1, anything else adds 0, which is exactly `paren_map`.

Two things have to be true: the counter can never go negative (that means a) with no (before it), and it has to end at 0 (otherwise some (never closed).

The end check is just `== 0`. For the negative check I return `None` and keep returning `None` once it happens. I need this to stick because `iterate` has no way to stop early, so a later (could push a negative counter back up to 0 and hide the problem. For example `['(', 'a', ')', ')', '(']` sums to 0 but is not matched, and the sticky `None` catches it.

- **3b.**

$$W(n) = W(n - 1) + O(1), \quad S(n) = S(n - 1) + O(1)$$

$$W(n) \in O(n), \quad S(n) \in O(n)$$

Work: `parens_update` is $O(1)$ and `iterate` calls it once per element.

Span: same as the work, because `iterate` is a chain. The counter after position i needs the counter after position $i - 1$, so there is no parallelism at all.

- **3c.**

Done in `main.py` as `parens_match_scan`:

```
counts = list(map(paren_map, mylist))
prefix_sums, total = scan(plus, 0, counts)
return reduce(min_f, 0, prefix_sums) >= 0 and total == 0
```

Same two conditions as 3a, just computed in parallel. The scan gives me all the prefix sums at once, and prefix sum i is the same counter value that 3a would have after reading position i . So instead of catching a negative counter as it happens, I compute all of them and take the min with reduce. The scan also hands me the total for free.

On ['(', 'a', ')', ')', '('] the prefix sums are $\langle 1, 1, 0, -1, 0 \rangle$, min is -1 , so it returns False even though the total is 0. That is why I need the min check and not just the total.

- **3d.**

$$W(n) = W(n/2) + O(n), \quad S(n) = S(n/2) + O(1)$$

$$W(n) \in O(n), \quad S(n) \in O(\log n)$$

These are for the scan. The map is $O(n)$ work and $O(1)$ span, and the reduce is $O(n)$ work and $O(\log n)$ span, so neither changes the totals.

Work: the contraction scan halves the input but the contract and expand steps each cost $O(n)$, so this is root-dominated and the top level dominates.

Span: the recursion has depth $\log n$ and each level only does $O(1)$ on the critical path.

Same work as 3b but the span drops from $O(n)$ to $O(\log n)$.

- **3e.**

Done in `main.py` as `parens_match_dc_helper`. Base cases: (is $(0, 1)$,) is $(1, 0)$, anything else and the empty list are $(0, 0)$.

For the merge, say the left half gives (i, j) and the right half gives (k, l) . After recursion each half always looks like some unmatched) followed by some unmatched (, because any) sitting after an unmatched (would have matched it already. So when I join the halves, the left half's j unmatched (land right before the right half's k unmatched), and they cancel:

$$matched = \min(j, k), \quad R = i + k - matched, \quad L = j + l - matched$$

The left half's i unmatched) and the right half's l unmatched (can never be fixed, so they carry through. This is $O(1)$.

Checking ((())): left is $(0, 3)$, right is $(3, 0)$, matched is 3, result $(0, 0)$, matched. Checking)(: left is $(1, 0)$, right is $(0, 1)$, matched is 0, result $(1, 1)$, not matched. That second one is why I need two numbers instead of one counter, since a single counter would sum to 0 and call it matched.

- **3f.**

$$W(n) = 2W(n/2) + O(1), \quad S(n) = S(n/2) + O(1)$$

$$W(n) \in O(n), \quad S(n) \in O(\log n)$$

Work: two calls on half the input and the merge is only $O(1)$, so the leaves dominate and it is linear.

Span: the two calls are independent and run in parallel, so the critical path follows one of them. Depth $\log n$ with $O(1)$ per level.

Same as 3c. That makes sense because the contraction scan is doing divide and conquer over the same structure.